Lesson 13: A Hit That Reacts

TIC-80

VIDEO GAME CREATION

This week: getting hit finally LOOKS like getting hit. A brief flash, then a small explosion, before the enemy disappears.

Open your own [YourName]Game project from Lesson 12 - don't start a new one.

Important: your score, lives, wave, and win/lose logic should all still work exactly as before. Today only changes how the enemy's removal LOOKS, not when your game-state actually changes.

Setup

1. Open your [YourName]Game project from Lesson 12.
2. Design a small explosion sprite in the Sprite Editor (a burst/star shape works well) in an empty slot, and a second "flash" version of your enemy sprite (the same silhouette, recolored bright white) in another empty slot - note both slot numbers.

Level 1 - Minimal (MVP)

On every hit, the enemy briefly flashes, and a small independent explosion effect appears and cleans itself up. Score, lives, wave, and win/lose logic are all unaffected.

3. TIC-80's code can't pause mid-frame, so this week's delay works differently: instead of removing a hit enemy from your enemies table immediately, give it a flash countdown field and remove it once that counter reaches 0.
4. In the bullets loop's hit branch, right after the existing sound/score/win-check lines, add e.flash = 6 to the enemy that was hit (code below) - do NOT remove it from the table yet.
5. Add a small explosions table, and table.insert an entry (with its own short countdown) at the hit position - this is independent of the enemy's own flash countdown.
6. Add a new update loop (runs every frame, before drawing): count down every enemy's flash field and every explosion's countdown, removing each from its table once it reaches 0.
7. In your drawing code, draw the flash sprite instead of the normal enemy sprite whenever e.flash is set, and draw each active explosion's sprite at its position.
8. Press Esc, type run, press Enter, then play through a full round - each hit should flash the enemy briefly, and a small burst should appear at that spot for a moment.

```
-- bullets loop's hit branch (revised):
score = score + 10
enemies_remaining = enemies_remaining - 1
```

```
sfx(2)
e.flash = 6
table.insert(explosions, {x=e.x, y=e.y, t=12})
if enemies_remaining <= 0 then
 if wave >= total_waves then
  game_active = false
  has_won = true
  sfx(3)
 else
  wave = wave + 1
  enemies_remaining = 5
  sfx(4)
  spawn_wave()
 end
end

-- new update loop (each frame, before drawing):
for i=#enemies,1,-1 do
 local e = enemies[i]
 if e.flash then
  e.flash = e.flash - 1
  if e.flash <= 0 then table.remove(enemies, i) end
 end
end
for i=#explosions,1,-1 do
 explosions[i].t = explosions[i].t - 1
 if explosions[i].t <= 0 then table.remove(explosions, i) end
end

-- drawing (per enemy):
if e.flash then spr(FLASH_SPRITE, e.x, e.y, 14) else spr(ENEMY_SPRITE,
e.x, e.y, 14) end
```

Level 2 - Complete (Alpha/Beta)

The flash and explosion play correctly on every single hit, with no leftover explosions, no enemies stuck mid-flash, and no change to score/lives/wave timing.

- Play a full round: every single hit shows a flash and an explosion, with no enemy left stuck mid-flash.
- Confirm score, wave-clear, and win/lose all still happen at exactly the moments they did last week - only each enemy's actual table.remove() is delayed, not the scoring logic.
- Confirm the explosions table never grows without bound - each entry's countdown reaches 0 and gets removed on its own.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: cycle the explosion through 2-3 different sprite frames across its 12-frame lifetime instead of one static sprite, for a simple hand-animated burst.
- Visual: instead of a flat flash sprite, alternate between the flash sprite and the normal sprite every couple of frames while e.flash is active, for a flicker instead of a solid flash.
- Structure: notice the explosions table knows nothing about score, lives, or waves - it's a fully self-contained piece of state, the same discipline as this course's bullets table.

If Something's Wrong

- Enemy vanishes instantly, no flash visible: confirm e.flash = 6 was actually added, and that the table.remove() for a hit enemy no longer happens in the hit branch itself.
- Enemy freezes on screen forever, never removed: confirm the new update loop is actually being called every frame from TIC(), and that it counts down and removes correctly.
- Explosion sprite never appears: confirm FLASH_SPRITE/explosion sprite slot numbers in your drawing code match what you actually drew in the Sprite Editor.

Want More?

Student Resource List > TIC-80 section, or github.com/nesbox/TIC-80/wiki